

1. Utilité du signal ML prédictif

Dans AdaptiveEngine, le ML sert à produire un **signal prédictif secondaire** :

Quelle est la probabilité estimée que cette recommandation aboutisse à une réussite ?

Concrètement, le modèle retourne :

```
{  
  "successProbability": 0.78,  
  "modelVersion": "local-rf-v1"  
}
```

Ce signal est utilisé dans **adaptive-engine-service** pour enrichir la recommandation avec :

```
"mlSuccessProbability": 0.78,  
"mlEnhancedScore": ...,  
"mlExplanation": "ML signal estimates the probability of successful recommendation  
based on historical recommendation traces."
```

Mais point très important pour ton mémoire :

Le ML ne remplace pas le moteur adaptatif rule-based.

Le moteur principal reste basé sur :

- les prérequis ;
- le diagnostic ;
- les traces ;
- le profil apprenant ;
- le score adaptatif ;
- les règles pédagogiques explicables.

Le ML est seulement une **aide expérimentale**.

Dans le code, le score ML est ajouté après la décision principale. Donc il ne décide pas directement :

```
PASS_DIAGNOSTIC
LEARN
REMEDICATION
COMPLETED
```

Il sert à enrichir l'explication et à préparer une future phase intelligente.

2. Où se trouve le pipeline ML ?

Les fichiers principaux sont dans :

```
ml-experiments/
```

Fichiers importants :

Fichier	Rôle
<code>generate_experimental_dataset.py</code>	génère un dataset synthétique expérimental
<code>run_first_ml_experiment.py</code>	entraîne et compare les modèles
<code>synthetic-recommendation-traces.json</code>	dataset synthétique brut
<code>synthetic-recommendation-traces-cleaned.csv</code>	dataset nettoyé utilisé pour l'entraînement
<code>metrics.json</code>	résultats des modèles
<code>feature_importance.csv</code>	importance des variables
<code>confusion_matrix.csv</code>	matrices de confusion
<code>model-serving/model.pkl</code>	modèle sauvegardé
<code>model-serving/app.py</code>	API FastAPI de prédiction

3. Pourquoi générer un dataset synthétique ?

Parce que le projet n'a pas encore assez de données réelles.

Dans un vrai système ML, il faudrait beaucoup de traces historiques réelles :

```
recommandation donnée
→ activité réalisée
→ résultat observé
→ concept réussi ou non
```

Mais dans ton PFE, la plateforme n'est pas encore déployée à grande échelle. Donc il n'y a pas suffisamment de **RecommendationTrace** réels labellisés.

Le dataset synthétique sert donc à :

1. valider techniquement le pipeline ML ;
2. vérifier que les features sont cohérentes ;
3. tester l'entraînement ;
4. tester le service FastAPI ;
5. tester l'intégration backend ;
6. préparer la future exploitation de vraies données.

Mais il faut écrire clairement :

```
Le dataset synthétique ne représente pas des apprenants réels.  
Il sert uniquement à valider expérimentalement le pipeline ML offline.
```

Dans le README ML, c'est formulé comme :

```
synthetic experimental dataset generated from AdaptiveEngine feature schema
```

4. Dataset utilisé

Dataset généré :

```
ml-experiments/synthetic-recommendation-traces.json  
ml-experiments/synthetic-recommendation-traces.csv
```

Dataset nettoyé pour entraînement :

```
ml-experiments/synthetic-recommendation-traces-cleaned.csv
```

Résumé réel observé dans **synthetic-dataset-summary.json** :

```
{  
  "seed": 42,  
  "rows": 260,  
  "targetDistribution": {  
    "True": 143,  
    "False": 107,  
    "None": 10  
  }  
}
```

```
}  
}
```

Donc :

Élément	Valeur
Lignes générées	260
Labels true	143
Labels false	107
Outcomes inconnus null	10
Lignes utilisées après nettoyage	250
Train	175
Test	75
Split	70/30
Stratification	Oui
random_state	42

Les 10 lignes **null** ne sont pas transformées artificiellement en **false**. Elles sont exclues de l'entraînement supervisé.

5. Profils simulés dans le dataset synthétique

Le générateur crée plusieurs profils pédagogiques cohérents avec AdaptiveEngine :

Profil	Idée simulée
HIGH_PERFORMER	bonne maîtrise, bons scores, peu de lacunes
AVERAGE_LEARNER	niveau moyen, résultats mixtes
STRUGGLING_LEARNER	faibles scores, lacunes, échecs répétés
REMEDIATION_SUCCESS	difficulté au départ puis réussite après remédiation
REMEDIATION_FAILURE	difficulté persistante malgré remédiation

Distribution réelle :

Profil	Nombre
AVERAGE_LEARNER	71
HIGH_PERFORMER	57
STRUGGLING_LEARNER	53

Profil	Nombre
REMIEDIATION_SUCCESS	40
REMIEDIATION_FAILURE	39

6. Target du modèle

La target finale est :

```
success
```

Dans le dataset, elle vient de :

```
conceptCompletedAfterRecommendation
```

Le script `run_first_ml_experiment.py` convertit :

```
conceptCompletedAfterRecommendation → success
```

Objectif :

```
Prédire si le concept recommandé sera complété après la recommandation.
```

Donc la question ML est :

```
Est-ce que cette recommandation a une forte probabilité de mener à la réussite du concept ?
```

7. Features utilisées pour l'entraînement

Les features réellement utilisées sont dans `run_first_ml_experiment.py`.

Features numériques

```
NUMERICAL_FEATURES = [  
    "adaptiveScore",  
    "prerequisiteScore",  
    "historicalPerformanceScore",  
    "pedagogicalOrderScore",
```

```
"engagementScore",  
"diagnosticWeaknessScore",  
"masteryScore",  
"averageAssessmentScore",  
"completedLabsCount",  
"tracesCount",  
]
```

Features catégorielles

```
CATEGORICAL_FEATURES = [  
    "profileType",  
    "recommendationType",  
]
```

Donc le modèle final utilise 12 features :

Feature	Type
adaptiveScore	numérique
prerequisiteScore	numérique
historicalPerformanceScore	numérique
pedagogicalOrderScore	numérique
engagementScore	numérique
diagnosticWeaknessScore	numérique
masteryScore	numérique
averageAssessmentScore	numérique
completedLabsCount	numérique
tracesCount	numérique
profileType	catégorielle
recommendationType	catégorielle

8. Comment chaque feature est calculée ou récupérée

adaptiveScore

Source :

```
AdaptivePathService.scoreConcept()
```

C'est le score rule-based principal :

```
0.35 prerequisiteScore
+ 0.25 diagnosticWeaknessScore
+ 0.15 historicalPerformanceScore
+ 0.15 pedagogicalOrderScore
+ 0.10 engagementScore
```

Il résume la pertinence pédagogique rule-based du concept recommandé.

prerequisiteScore

Source :

```
scoreBreakdown.prerequisiteScore
```

Calculé dans [AdaptivePathService](#).

Logique :

Situation	Valeur
prérequis manquant	0.0
prérequis déclarés et satisfaits	1.0
aucun prérequis bloquant	0.5

Utilité ML :

```
Indique si le concept est pédagogiquement accessible.
```

diagnosticWeaknessScore

Source :

```
scoreBreakdown.diagnosticWeaknessScore
```

Logique :

Situation	Valeur
concept échoué dans diagnostic	1.0

Situation	Valeur
prérequis échoué	0.7
pas de faiblesse directe	0.3

Utilité ML :

Indique si le concept est lié à une lacune diagnostiquée.

historicalPerformanceScore

Source :

scoreBreakdown.historicalPerformanceScore

Calculé à partir des traces `scoreObtenu`.

Logique :

Moyenne historique	Valeur
aucune trace	0.5
moyenne $\geq$ 80	1.0
moyenne $\geq$ 60	0.7
moyenne $<$ 60	0.4

Utilité ML :

Résume les performances passées de l'apprenant.

pedagogicalOrderScore

Source :

scoreBreakdown.pedagogicalOrderScore

Calculé selon la position du concept dans l'ordre pédagogique :


```
1 - position / totalConcepts
```

avec minimum :

```
0.1
```

Utilité ML :

Indique si le concept est proche du début/logique actuelle du parcours.

engagementScore

Source :

```
scoreBreakdown.engagementScore
```

Calculé depuis les traces et labs.

Logique simplifiée :

Situation	Valeur
aucune activité	0.5
trace récente	0.7
lab complété	1.0
plusieurs échecs ou labs abandonnés	0.3

Utilité ML :

Approxime l'engagement de l'apprenant.

masteryScore

Source :

```
learnerProfile.masteryScore
```

Calculé dans `buildLearnerProfile()`.

Il synthétise le niveau global de maîtrise de l'apprenant à partir :

- du diagnostic ;
- des scores de traces ;
- des concepts maîtrisés.

Utilité ML :

Indique le niveau global de l'apprenant.

averageAssessmentScore

Source :

`learnerProfile.averageAssessmentScore`

Calculé comme moyenne des scores d'évaluations/traces.

Utilité ML :

Mesure la performance moyenne aux évaluations.

completedLabsCount

Source :

`learnerProfile.completedLabsCount`

Calculé à partir des soumissions TP/labs avec statut complété.

Utilité ML :

Mesure l'activité pratique réalisée.

tracesCount

Source :

```
learnerProfile.tracesCount
```

Nombre de traces d'apprentissage disponibles.

Utilité ML :

```
Mesure la quantité d'historique disponible sur l'apprenant.
```

profileType

Source :

```
learnerProfile.profileType
```

Valeurs possibles côté moteur :

```
DATA_INSUFFICIENT  
NEEDS_REMEDIATION  
HIGH_PERFORMING  
PROGRESSING
```

Dans le dataset synthétique nettoyé, on retrouve principalement :

```
HIGH_PERFORMING  
PROGRESSING  
NEEDS_REMEDIATION
```

Utilité ML :

```
Donne une catégorie globale du profil apprenant.
```

recommendationType

Source :

Dans le dataset, elle est normalisée depuis :

```
recommendationType  
ou recommendationContext  
ou nextAction
```

Logique du script :

```
if nextAction == "LEARN":  
    recommendationType = "NORMAL_PROGRESS"  
elif nextAction == "PASS_DIAGNOSTIC":  
    recommendationType = "DIAGNOSTIC"  
elif nextAction == "COMPLETED":  
    recommendationType = "VALIDATION"  
else:  
    recommendationType = nextAction
```

Valeurs possibles :

```
NORMAL_PROGRESS  
REMEDICATION  
DIAGNOSTIC  
VALIDATION
```

Utilité ML :

```
Indique le contexte de la recommandation.
```

9. Features exclues pour éviter le data leakage

Très important scientifiquement : certaines colonnes existent dans le dataset mais ne sont pas utilisées comme features, car elles représentent le futur ou le résultat après recommandation.

Exclues dans `run_first_ml_experiment.py` :

```
EXCLUDED_OUTCOME_FIELDS = [  
    "quizScoreAfterRecommendation",  
    "conceptCompletedAfterRecommendation",  
    "success",  
    "remediationSucceeded",  
    "outcomeCapturedAt",  
    "conceptCompleted",  
    "labSubmittedAfterRecommendation",  
    "learnerDropped",  
    "recommendationAccepted",
```

```
"lastActivityScore",  
"remediationSuccess",  
]
```

Pourquoi ?

Parce que ces variables donnent directement ou indirectement le résultat qu'on veut prédire.

Exemple :

```
quizScoreAfterRecommendation
```

ne doit pas être utilisé pour prédire :

```
conceptCompletedAfterRecommendation
```

car c'est une information future.

Donc le pipeline évite une erreur méthodologique importante.

10. Prétraitement du dataset

Dans `run_first_ml_experiment.py`, les features sont séparées en deux groupes.

Pour les variables numériques

Pipeline :

```
SimpleImputer(strategy="median")  
StandardScaler()
```

Donc :

1. valeurs manquantes remplacées par la médiane ;
2. normalisation des valeurs numériques.

Pour les variables catégorielles

Pipeline :

```
SimpleImputer(strategy="most_frequent")  
OneHotEncoder(handle_unknown="ignore")
```

Donc :

1. valeurs manquantes remplacées par la modalité la plus fréquente ;
2. encodage one-hot.

Exemple :

```
profileType = HIGH_PERFORMING
```

devient une colonne binaire :

```
profileType_HIGH_PERFORMING
```

11. Entraînement : procédure exacte

Commande utilisée :

```
python ml-experiments/run_first_ml_experiment.py --source synthetic
```

Le script fait :

1. charge le dataset synthétique ;
2. convertit `conceptCompletedAfterRecommendation` en `success` ;
3. enlève les lignes sans label ;
4. garde les features autorisées ;
5. applique le preprocessing ;
6. découpe le dataset :

```
train_test_split(  
    test_size=0.3,  
    random_state=42,  
    stratify=y  
)
```

Donc :

Partie	Nombre
Train	175
Test	75

7. entraîne trois modèles ;

8. évalue les modèles ;
9. sélectionne le meilleur ;
10. sauvegarde le modèle dans :

```
ml-experiments/model-serving/model.pkl
```

12. Modèles testés

Trois modèles sont réellement testés :

1. Dummy Classifier

```
DummyClassifier(strategy="most_frequent", random_state=42)
```

Utilité :

```
Baseline naïve.
```

Il prédit toujours la classe majoritaire.

2. Logistic Regression

```
LogisticRegression(  
    max_iter=1000,  
    class_weight="balanced",  
    random_state=42  
)
```

Utilité :

```
Modèle simple, linéaire, interprétable.
```

3. Random Forest

```
RandomForestClassifier(  
    n_estimators=250,  
    random_state=42,  
    class_weight="balanced",
```

```
    min_samples_leaf=2  
)
```

Utilité :

Modèle non linéaire capable de capturer des interactions entre les features.

13. Résultats obtenus

Résultats réels dans `metrics.json`.

Modèle	Accuracy	Precision	Recall	F1	ROC-AUC
Dummy most frequent	0.573	0.573	1.000	0.729	0.500
Logistic Regression	0.707	0.784	0.674	0.725	0.777
Random Forest	0.720	0.806	0.674	0.734	0.749

Matrice de confusion Random Forest :

```
TN = 25  
FP = 7  
FN = 14  
TP = 29
```

Sous forme matrice :

```
[[25, 7],  
 [14, 29]]
```

14. Pourquoi Random Forest a été choisi ?

Le script choisit le meilleur modèle avec ce critère :

```
best_name = max(results, key=lambda key: (results[key]["f1"], results[key]  
["roc_auc"] or -1))
```

Donc le critère principal est :

F1-score

puis :

ROC-AUC

En résultats :

Modèle	F1
Dummy	0.729
Logistic Regression	0.725
Random Forest	0.734

Le meilleur F1 est :

Random Forest = 0.734

Donc le modèle sauvegardé est :

```
"bestModel": "random_forest"
```

et la version :

```
"modelVersion": "local-rf-v1"
```

Fichier sauvegardé :

```
ml-experiments/model-serving/model.pkl
```

Pourquoi ce choix est défendable ?

Random Forest est défendable parce que :

1. il obtient le meilleur F1-score dans cette expérience ;
2. il obtient la meilleure accuracy ;
3. il obtient la meilleure precision ;
4. il gère bien les relations non linéaires ;

5. il fournit une importance des variables ;
6. il est plus flexible qu'une régression logistique.

Mais il faut être prudent :

La Logistic Regression a un ROC-AUC légèrement supérieur.

Donc il ne faut pas dire :

Random Forest est absolument le meilleur modèle.

Il faut dire :

Random Forest a été retenu dans cette expérimentation car il obtient le meilleur F1-score et une meilleure précision globale sur le test set synthétique, selon le critère de sélection implémenté.

15. Importance des variables

Top features Random Forest dans `feature_importance.csv` :

Feature	Importance
averageAssessmentScore	0.2149
masteryScore	0.1692
engagementScore	0.1309
adaptiveScore	0.0875
pedagogicalOrderScore	0.0822
historicalPerformanceScore	0.0669
completedLabsCount	0.0667
tracesCount	0.0530
diagnosticWeaknessScore	0.0278
profileType_HIGH_PERFORMING	0.0259

Interprétation :

Le modèle donne plus d'importance aux variables liées :

- aux scores d'évaluation ;

- à la maîtrise globale ;
- à l'engagement ;
- au score adaptatif rule-based ;
- à l'ordre pédagogique.

C'est cohérent pédagogiquement : un apprenant avec bonne maîtrise, bon historique et bon engagement a plus de chance de réussir après une recommandation.

16. Comment le modèle est utilisé ensuite

Le modèle est chargé dans :

```
ml-experiments/model-serving/app.py
```

Endpoint :

```
POST /api/ml/predict-success
```

Payload attendu :

```
{
  "adaptiveScore": 0.82,
  "prerequisiteScore": 0.9,
  "historicalPerformanceScore": 0.75,
  "pedagogicalOrderScore": 0.8,
  "engagementScore": 0.7,
  "diagnosticWeaknessScore": 0.3,
  "masteryScore": 0.65,
  "averageAssessmentScore": 72,
  "completedLabsCount": 5,
  "tracesCount": 18,
  "profileType": "INTERMEDIATE",
  "recommendationType": "NORMAL_PROGRESS"
}
```

Réponse :

```
{
  "successProbability": 0.78,
  "modelVersion": "local-rf-v1"
}
```

Dans `adaptive-engine-service`, cette probabilité est combinée avec le score rule-based :

```
mlEnhancedScore =  
0.8 × adaptiveScore  
+ 0.2 × mlSuccessProbability
```

Mais encore une fois :

La décision principale reste rule-based.

17. Exemple d'explication mémoire

Tu peux écrire :

Le module ML d'AdaptiveEngine a été conçu comme un signal prédictif secondaire. Son objectif n'est pas de remplacer le moteur adaptatif explicable, mais d'estimer la probabilité qu'une recommandation conduise à la réussite du concept recommandé. Pour entraîner ce premier modèle expérimental, nous avons utilisé le schéma des traces `RecommendationTrace`, qui regroupe des variables issues du moteur adaptatif, du profil apprenant et de l'historique d'apprentissage. En l'absence d'un volume suffisant de traces réelles labellisées, un dataset synthétique contrôlé a été généré afin de valider le pipeline offline d'entraînement, de prétraitement, d'évaluation et de serving. Ce dataset est explicitement présenté comme expérimental et non comme une preuve d'efficacité réelle.

Puis :

Trois modèles ont été comparés : un classifieur naïf majoritaire, une régression logistique et une forêt aléatoire. Le modèle Random Forest a été retenu car il obtient le meilleur F1-score sur le test set synthétique, avec une précision et une exactitude supérieures aux autres modèles. Le modèle est ensuite sauvegardé sous forme de pipeline sérialisé dans `model1.pkl` et exposé via une API FastAPI. Lorsqu'il est disponible, le moteur adaptatif l'interroge pour obtenir une probabilité de réussite, utilisée comme signal complémentaire. En cas d'indisponibilité du service ML, le moteur revient automatiquement au fonctionnement rule-based.

18. Formulation prudente à dire devant le jury

Version orale :

Dans mon projet, le ML n'est pas le moteur principal. Le moteur principal reste un moteur rule-based explicable. Le ML sert uniquement à estimer une probabilité de réussite après recommandation. Comme nous n'avons pas encore assez de traces réelles, j'ai généré un dataset synthétique contrôlé à partir du schéma `RecommendationTrace`. Ce dataset reproduit différents profils pédagogiques : apprenant performant, moyen, en difficulté, réussite ou échec de remédiation. Ensuite, j'ai entraîné trois modèles : Dummy Classifieur, Logistic Regression et Random Forest. Random Forest a été retenu parce qu'il obtient le meilleur F1-score dans cette expérimentation. Le résultat est exposé par un service FastAPI, mais il

reste un signal secondaire : si le service ML est indisponible, le moteur adaptatif continue à fonctionner normalement avec ses règles explicables.

Version courte :

Le ML sert à prédire la probabilité de réussite d'une recommandation. Il a été entraîné offline sur un dataset synthétique contrôlé basé sur les traces RecommendationTrace. Les features viennent du score adaptatif, du profil apprenant, des traces et des prérequis. Random Forest a été choisi car il obtient le meilleur F1-score parmi les modèles testés, mais il reste un signal secondaire : le moteur principal reste rule-based.